Exercise 1 — Final Report

Real-Time Sound Classification Using Microphone Input, FFT Feature Extraction, OSC Communication, and KNN Classification in Wekinator

Music and Multimedia Streaming Lab

Abstract. This exercise implements a real-time sound classification system using Processing and Wekinator. The system captures live audio from a microphone, extracts numerical audio features using the Fast Fourier Transform (FFT), sends those features to Wekinator through OSC, trains a K-Nearest Neighbors classifier, and visualizes the predicted class in a second Processing sketch. The implemented and tested configuration uses **12 reduced FFT bands** as input features, a **KNN classifier with** $k = 3$, and two vocal sound classes: `eeeeeeee` (Class 1) and `uuuuuuuu` (Class 2). The output sketch displays a red canvas for Class 1 and a blue canvas for Class 2. The project demonstrates the complete workflow of an interactive machine learning system: real-time sensing, feature extraction, OSC communication, machine learning classification, and real-time visual feedback.

Tested System Configuration

Feature mode	3 — 12 reduced FFT bands
Wekinator inputs	12
Wekinator outputs	1
Output type	Classification
Classifier	K-Nearest Neighbours, $k = 3$
Class 1	`eeeeeeeeeeee` → red canvas
Class 2	`uuuuuuuuuuuu` → blue canvas
Total training examples	1081 (560 class 1 / 521 class 2)

OSC Communication

Direction	OSC address	Port
Input sketch → Wekinator	`/wek/inputs`	`6448`
Wekinator → Output sketch	`/wek/outputs`	`12000`

Compiled with LaTeX on Overleaf

Contents

1 Objective of the Exercise **ii**

2 Tools and Libraries Used **ii**
2.1 Processing . . . ii
2.2 Processing Sound Library . . . ii
2.3 oscP5 Library . . . ii
2.4 Wekinator . . . iii

3 Complete System Architecture **iii**

4 Input Processing Sketch **iii**
4.1 Purpose . . . iii
4.2 Audio Input . . . iii
4.3 FFT Analysis . . . iv
4.4 Meaning of FFT . . . iv
4.5 Feature Modes . . . iv
4.6 Downsampling from 512 to 12 Bands . . . iv
4.7 OSC Configuration of the Input Sketch . . . v

5 Wekinator Project **v**
5.1 Input Settings . . . v
5.2 Output Settings . . . v
5.3 Why 1 Output but 2 Classes . . . v
5.4 Class Definitions . . . vi

6 KNN Classifier **vi**
6.1 What KNN Is . . . vi
6.2 KNN with $k = 3$ . . . vi

7 Training Data **vi**

8 Training Process **vi**

9 Output Processing Sketch **vii**
9.1 Purpose . . . vii
9.2 OSC Receiving Settings . . . vii
9.3 Class-to-Color Mapping . . . viii
9.4 Why the Output Sketch Is Separate . . . viii

10 Correct Running Order **viii**
10.1 During Training . . . viii
10.2 During Testing and Demonstration . . . viii

11 Final Tested Configuration **ix**

12 Results **ix**

13 Discussion **ix**
13.1 Importance of Feature Extraction . . . x
13.2 Feature Dimensionality Comparison . . . x
13.2.1 Full FFT — 512 Bands . . . x
13.2.2 Reduced FFT — 64 Bands . . . x

13.2.3 Reduced FFT — 12 Bands (current experiment) x
13.2.4 Mean and Variance — 2 Values x
13.3 Spectral Features vs Statistical Features x

14 Common Problems and Debugging **xi**
14.1 Wekinator Does Not Receive Inputs xi
14.2 Wekinator Predicts the Wrong Class xi
14.3 Output Sketch Does Not Change Color xi

15 What Was Learned **xi**
15.1 Real-Time Audio Processing xi
15.2 FFT as Feature Extraction xi
15.3 Machine Learning Workflow xi
15.4 OSC Communication xii
15.5 Interactive System Design xii

16 Deliverables **xii**
16.1 Input Processing Sketch xii
16.2 Output Processing Sketch xii
16.3 Wekinator Project xii
16.4 Report xii

17 Final Conclusion **xii**

1. Objective of the Exercise

The objective of this exercise is to design and develop an interactive system capable of classifying live sounds in real time.

The system must:

1. Capture live audio from a microphone.
2. Analyze the sound using FFT.
3. Extract useful numerical features from the audio signal.
4. Send those features from Processing to Wekinator using OSC.
5. Train a machine learning classifier in Wekinator.
6. Use the trained classifier to recognize two different sound classes.
7. Send the predicted class from Wekinator to a second Processing sketch.
8. Visualize the predicted class by changing the canvas color.

The exercise also requires comparing four different feature representations:

- Full FFT with 512 bands.
- Reduced FFT with 64 bands.
- Reduced FFT with 12 bands.
- Mean and variance of the 512 FFT bands.

In the current completed version, the system was implemented and tested using the **12-band FFT representation**.

2. Tools and Libraries Used

2.1 Processing

Processing is used to create the two sketches required by the exercise:

1. **Input sketch** — captures microphone audio, extracts features, and sends them to Wekinator.
2. **Output sketch** — receives the predicted class from Wekinator and changes the screen color.

2.2 Processing Sound Library

Used in the input sketch to provide the two key audio classes:

```
AudioIn mic;    // captures live microphone sound
FFT fft;        // converts audio to frequency-domain representation
```

2.3 oscP5 Library

Used for OSC (Open Sound Control) communication between Processing and Wekinator:

```
Processing input sketch  /wek/inputs →  Wekinator  /wek/outputs →  Processing output
sketch
```

2.4 Wekinator

Wekinator is the machine learning component. It receives feature vectors from the input sketch, records training examples for each class, trains a classifier, and sends the predicted class to the output sketch. In this exercise it is configured as a **two-class classifier** using **K-Nearest Neighbors**.

3. Complete System Architecture

The system contains three connected parts: the Processing input sketch, the Wekinator classifier, and the Processing output sketch.

```
User makes a sound
↓
Microphone captures audio
↓
Processing input sketch receives audio
↓
FFT analysis extracts frequency features
↓
Processing sends features via /wek/inputs to port 6448
↓
Wekinator classifies the sound
↓
Wekinator sends predicted class via /wek/outputs to port 12000
↓
Processing output sketch receives class
↓
Canvas color changes
```

The system creates a complete real-time interaction loop:

```
human sound → audio analysis → machine learning → visual feedback
```

4. Input Processing Sketch

4.1 Purpose

The input sketch handles the first half of the pipeline:

```
microphone audio → feature extraction → Wekinator input
```

It does not classify the sound by itself — it only prepares and sends numerical data to Wekinator. Its tasks are: start the microphone, compute a 512-band FFT, select the feature representation, send the feature vector via OSC, and display a real-time feature visualization.

4.2 Audio Input

```
mic = new AudioIn(this, 0);
mic.start();
```

The argument `0` selects the first available audio input device. `mic.start()` begins capturing live sound.

4.3 FFT Analysis

```
int bands = 512;
float[] spectrum = new float[bands];

fft = new FFT(this, bands);
fft.input(mic);

// Inside draw():
fft.analyze(spectrum);
```

After every call to `fft.analyze(spectrum)`, the array `spectrum[0..511]` holds the current energy at each of the 512 frequency bands: low frequencies at index 0, high frequencies at index 511.

4.4 Meaning of FFT

FFT (Fast Fourier Transform) converts a time-domain audio signal into a frequency-domain representation. Instead of describing how air pressure changes over time, the FFT answers:

> Which frequencies are present in this sound, and how strong is each one?

Different sounds produce different spectral patterns. The vocal sound `eeeeeeee` and `uuuuuuuu` have different frequency shapes because the mouth shape and vocal resonance differ. The classifier can learn these differences from FFT features.

4.5 Feature Modes

`featureMode`	Feature representation	Wekinator inputs
1	Full FFT	512
2	Reduced FFT	64
3	Reduced FFT	12
4	Mean + variance	2

The current implemented project uses:

```
int featureMode = 3;
```

4.6 Downsampling from 512 to 12 Bands

```
512 detailed FFT bands
↓
Group neighboring frequency bands
↓
Calculate the average of each group
↓
12 broad frequency features
```

Proportional indexing ensures all 512 bins contribute to the 12 output bands:

```
int start = floor(i * bands / float(targetBands));
int end   = floor((i + 1) * bands / float(targetBands));
```

This avoids ignoring remainder bins (since 512/12 is not exact). Advantages of 12-band mode: simpler Wekinator setup, faster training, easier debugging, lower computational cost. Disadvantage: some fine frequency detail is lost.

4.7 OSC Configuration of the Input Sketch

```
OSC message address  : /wek/inputs
Destination host     : 127.0.0.1   (localhost)
Destination port     : 6448
```

The sketch sends a 12-float OSC message approximately every 30 ms (≈33 messages/second):

```
OscMessage msg = new OscMessage("/wek/inputs");
for (int i = 0; i < features.length; i++) msg.add(features[i]);
oscP5.send(msg, wekinator);
```

A conceptual example of one message in mode 3:

```
/wek/inputs  f1  f2  f3  f4  f5  f6  f7  f8  f9  f10  f11  f12
```

Processing sends 12 features, so Wekinator must be configured with 12 inputs. A mismatch causes silent failures — dimensions are wrong but no error is shown.

5. Wekinator Project

5.1 Input Settings

```
Input port          : 6448
Input OSC message   : /wek/inputs
Number of inputs    : 12
```

5.2 Output Settings

```
Output OSC message  : /wek/outputs
Output host         : localhost
Output port         : 12000
Number of outputs   : 1
Output type         : Classification
Number of classes   : 2
```

5.3 Why 1 Output but 2 Classes

Wekinator sends one number that encodes the predicted class label:

Output value	Meaning
1.0	Class 1 — eeeeeeeeeeee
2.0	Class 2 — uuuuuuuuuuuu

One output is sufficient to encode a multi-class label in Wekinator's classification mode.

5.4 Class Definitions

Class	Sound recorded	Visual result
1	`eeeeeeeeeeeee`	Red canvas
2	`uuuuuuuuuuuuu`	Blue canvas

6. KNN Classifier

6.1 What KNN Is

KNN (K-Nearest Neighbors) classifies a new input by comparing it to all stored training examples. It identifies the k closest examples and predicts the class that appears most often among them.

6.2 KNN with $k = 3$

```
Nearest example 1 -> Class 1
Nearest example 2 -> Class 1
Nearest example 3 -> Class 2
---------------
Majority vote -> Class 1 (2 out of 3)
```

With $k = 1$, a single noisy training example can dominate the prediction. With $k = 3$, the decision is a local majority vote — more stable and less sensitive to outliers in the training data.

After changing k, the model must be retrained. Existing recordings do not need to be deleted — only the training step must be repeated.

7. Training Data

Category	Count
Total training examples	1081
Class 1 examples	560
Class 2 examples	521

The dataset is well balanced: 560 vs 521 means neither class dominates. An imbalanced dataset can cause the classifier to favor the majority class, reducing accuracy for the minority class.

8. Training Process

1. **Run the input sketch.**
   The input sketch must run first. Wekinator does not access the microphone directly — it

receives only the feature vectors sent by Processing. Without the input sketch, Wekinator receives no data.

2. **Create or open the Wekinator project.**
   Configure with 12 inputs, 1 output, 2 classes, classifier output type, input port `6448`, output port `12000`.

3. **Record Class 1.**
   Set the class dropdown to `1`. Say `eeeeeeeeeeeee` while pressing **Start Recording**. Press **Stop Recording** when done.

4. **Record Class 2.**
   Change the dropdown to `2`. Say `uuuuuuuuuuuuu` while recording.

5. **Repeat recordings for both classes.**
   Multiple sessions per class help Wekinator generalize across variations in loudness, pitch, microphone distance, and background noise.

6. **Set KNN to $k = 3$.**
   Open model settings in Wekinator, select K-Nearest Neighbor, set neighbors to `3`.

7. **Train the model.**
   Click **Train**. Wekinator builds the classifier from all recorded examples.

8. **Run the model.**
   Click **Run**. Wekinator now classifies live input in real time.

After training, the correct action is to click **Run** — not **Start Recording**. Pressing Start Recording during testing would add unwanted examples to the dataset.

9. Output Processing Sketch

9.1 Purpose

The output sketch handles visualization only:

```
Receive Wekinator output → change background color
```

It does not analyze sound, does not compute FFT, and does not train the model.

9.2 OSC Receiving Settings

```
oscP5 = new OscP5(this, 12000);
```

The sketch listens on port `12000` for messages at `/wek/outputs`, matching the Wekinator output configuration exactly.

9.3 Class-to-Color Mapping

State	Sound	Canvas
Class 1	`eeeeeeeeeeeee`	Red `(255, 0, 0)`
Class 2	`uuuuuuuuuuuuu`	Blue `(0, 0, 255)`
Waiting / unknown	—	Dark gray `(80)`

9.4 Why the Output Sketch Is Separate

The project uses two Processing sketches because each has a distinct responsibility:

Sketch	Responsibility
Input sketch	Microphone input and feature extraction
Output sketch	Visual feedback from Wekinator prediction

This separation follows the Wekinator workflow and makes each component easier to understand, test, and debug independently.

10. Correct Running Order

10.1 During Training

Component	Required?
Input Processing sketch	Yes
Wekinator	Yes
Output Processing sketch	Not needed

10.2 During Testing and Demonstration

All three parts must run simultaneously:

```
Class 1 flow:
User says "eeeeeeeeeeeee"
↓
Input sketch sends 12 FFT features via /wek/inputs
↓
Wekinator predicts Class 1, sends /wek/outputs 1.0
↓
Output sketch canvas turns red

Class 2 flow:
User says "uuuuuuuuuuuuu"
↓
Input sketch sends 12 FFT features via /wek/inputs
↓
Wekinator predicts Class 2, sends /wek/outputs 2.0
↓
Output sketch canvas turns blue
```

11. Final Tested Configuration

Complete Final Configuration

Input sketch feature mode	3
Feature representation	12 reduced FFT bands
Input OSC message	`/wek/inputs`
Input destination port	`6448`
Wekinator input count	12
Wekinator output count	1
Wekinator output type	Classification
Number of classes	2
Classifier	K-Nearest Neighbors
k	3
Output OSC message	`/wek/outputs`
Output destination port	`12000`
Output sketch visualization	Color only
Class 1 color	Red `(255, 0, 0)`
Class 2 color	Blue `(0, 0, 255)`
Class 1 sound	`eeeeeeeeeeeee`
Class 2 sound	`uuuuuuuuuuuuu`
Total training examples	1081 (560 / 521)

12. Results

After training, Wekinator was switched to Run mode and the complete three-part system was tested.

When the user said `eeeeeeeeeeeee`, Wekinator predicted Class 1 and the output sketch displayed a **red** canvas.

When the user said `uuuuuuuuuuuuu`, Wekinator predicted Class 2 and the output sketch displayed a **blue** canvas.

This confirms:

- The OSC communication chain works correctly in both directions.
- The KNN classifier with $k = 3$ successfully distinguishes the two vocal classes using 12 reduced FFT features.
- The output sketch correctly maps the predicted class to a canvas color in real time.

13. Discussion

13.1 Importance of Feature Extraction

The classifier never receives raw audio. It receives structured numerical feature vectors. FFT features are well suited to this task because they describe the frequency content of sound, and the vowels `eeeeeeee` and `uuuuuuuu` differ primarily in their spectral shape — the resonant formant frequencies of the vocal tract.

13.2 Feature Dimensionality Comparison

Mode	Type	# inputs	Summary
1	Full FFT	512	Maximum spectral detail; higher training complexity; noise-sensitive
2	Reduced FFT	64	Good balance; retains spectral shape; moderate complexity
3	Reduced FFT	12	Simple; fast; sufficient for clearly distinct vowel classes
4	Mean + variance	2	Very compact; loses spectral shape; weak for vowel classification

13.2.1 Full FFT — 512 Bands

Advantages: highest spectral detail; best for subtle sound differences.
Disadvantages: high dimensionality; may require more training examples; more sensitive to noise.

13.2.2 Reduced FFT — 64 Bands

Advantages: retains most of the spectral shape; easier to train than 512 bands.
Disadvantages: loses some fine frequency detail.

13.2.3 Reduced FFT — 12 Bands (current experiment)

Advantages: simple to configure; fast training; easier to debug; low risk of overfitting.
Disadvantages: loses detailed frequency information; may fail for very similar sounds.
Result: sufficient to distinguish `eeeeeeee` from `uuuuuuuu` in this experiment.

13.2.4 Mean and Variance — 2 Values

Advantages: extremely compact; fast computation; very easy to understand.
Disadvantages: loses almost all spectral shape; insufficient for vowel classification where frequency distribution is the primary distinguishing feature.

13.3 Spectral Features vs Statistical Features

Type	Examples	What is captured
Spectral	512, 64, or 12 FFT bands	Shape of frequency distribution
Statistical	Mean, variance	Summary statistics of the spectrum

Spectral features are more appropriate for this task because vowel sounds differ in formant structure — a property visible in the spectral shape but not in mean and variance alone.

14. Common Problems and Debugging

14.1 Wekinator Does Not Receive Inputs

- Input sketch is not running.
- Wekinator input port is not `6448`.
- OSC message is not `/wek/inputs`.
- Wekinator input count does not match `features.length`.
- Microphone is not working or muted.
- Processing Sound library is not installed.

14.2 Wekinator Predicts the Wrong Class

- Too few training examples recorded.
- Class recordings are not clear or consistent.
- Background noise is high.
- Microphone distance changed between training and testing.
- KNN k value was changed but model was not retrained.

14.3 Output Sketch Does Not Change Color

- Wekinator is not in **Run** mode.
- Output sketch is not running.
- Wekinator output port is not `12000`.
- Wekinator output message is not `/wek/outputs`.
- Another program is already using port `12000`.
- Check the Processing console — `println` should print received class values.

15. What Was Learned

15.1 Real-Time Audio Processing

Processing can capture microphone audio and compute FFT continuously inside the `draw()` loop, enabling real-time feature extraction at approximately 33 frames per second.

15.2 FFT as Feature Extraction

FFT converts raw audio into frequency features that are both compact and informative. For vocal sounds, these features reflect the formant structure of the voice — a key discriminant between different vowel classes.

15.3 Machine Learning Workflow

```
Record examples → Train model → Run model → Test live classification
```

Changing a model setting (such as k) requires re-training. Training examples do not need to be re-recorded.

15.4 OSC Communication

OSC allows independent programs to exchange data over the local network in real time. The input sketch sends features to Wekinator; Wekinator sends predictions to the output sketch. Each program handles one part of the pipeline independently.

15.5 Interactive System Design

The project creates a complete interaction loop: a human sound immediately produces a visual response. Wekinator handles the intelligence; Processing handles sensing and display. Together they form a modular and easily extensible system.

16. Deliverables

16.1 Input Processing Sketch

Captures microphone audio, extracts FFT features, sends data to Wekinator.

Key elements: `AudioIn`, `FFT` with 512 bands, `featureMode = 3` (12-band reduced FFT), OSC address `/wek/inputs`, port `6448`.

16.2 Output Processing Sketch

Receives Wekinator predictions and changes the canvas color.

Key elements: OSC port `12000`, address `/wek/outputs`, red for Class 1, blue for Class 2.

16.3 Wekinator Project

Trained KNN classifier: 12 inputs, 1 output, 2 classes, $k = 3$, 1081 training examples.

16.4 Report

This document — explains system architecture, configuration, training, results, and discussion.

17. Final Conclusion

This exercise successfully implements a complete real-time sound classification system.

The Processing input sketch captures live microphone audio and extracts 12 reduced FFT features using downsampling from a 512-band FFT analysis. These features are sent to Wekinator every 30 ms using the OSC message `/wek/inputs` on port `6448`.

Wekinator uses a K-Nearest Neighbors classifier with $k = 3$ to classify the sound into two classes trained on the vocal sounds `eeeeeeee` and `uuuuuuuu`, using a balanced dataset of 1081 total examples. The predicted class is sent via OSC on `/wek/outputs` to port `12000`.

The Processing output sketch receives this prediction and immediately changes the background color — red for Class 1, blue for Class 2.

Full Interactive Pipeline

```
live microphone sound
↓
FFT feature extraction (12 reduced bands)
↓
KNN classification in Wekinator (k = 3)
↓
real-time color visualization (red / blue)
```

Switching to another configuration requires only two changes: set `featureMode` in the input sketch to `1`, `2`, or `4`, and create a matching Wekinator project with the corresponding number of inputs (512, 64, or 2). All other components — OSC addresses, ports, output sketch — remain identical.

The project fulfills all main requirements of Exercise 1 and demonstrates the core principles of interactive machine learning: real-time sensing, feature extraction, OSC communication, supervised learning, and real-time visual feedback.